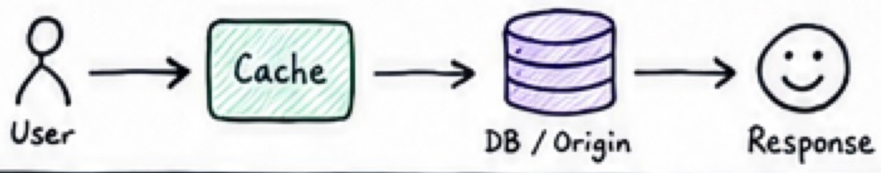
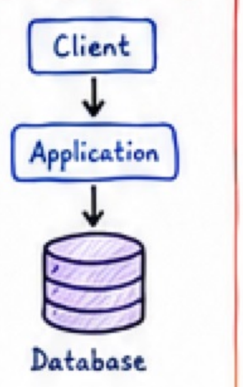
BACKEND CACHE DESIGN – COMPLETE GUIDE

Store the right data, at the right place, for the right time.



1 WHY DO WE NEED CACHING?

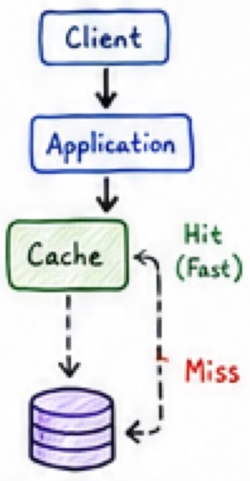
Without Cache



Every request hits DB

- ✗ High Latency
- ✗ High DB Load
- ✗ High Cost

With Cache



- ✓ Low Latency
- ✓ Less DB Load
- ✓ Better Scalability
- ✓ Better UX
- ✓ Lower Cost

When to cache?

- ✓ Read >> Write
- ✓ Expensive to compute/fetch
- ✓ Same data requested again & again
- ✓ Stale data can be tolerated

Don't cache!

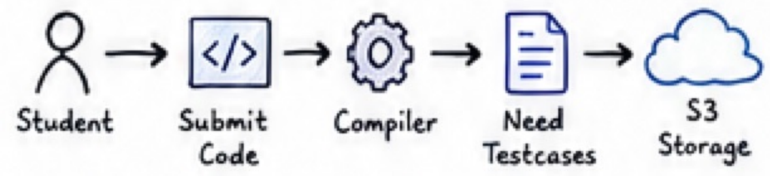
- ✗ OTP, Tokens
- ✗ Payment / Banking
- ✗ Live Stock Prices
- ✗ Frequently changing data
- ✗ Sensitive information

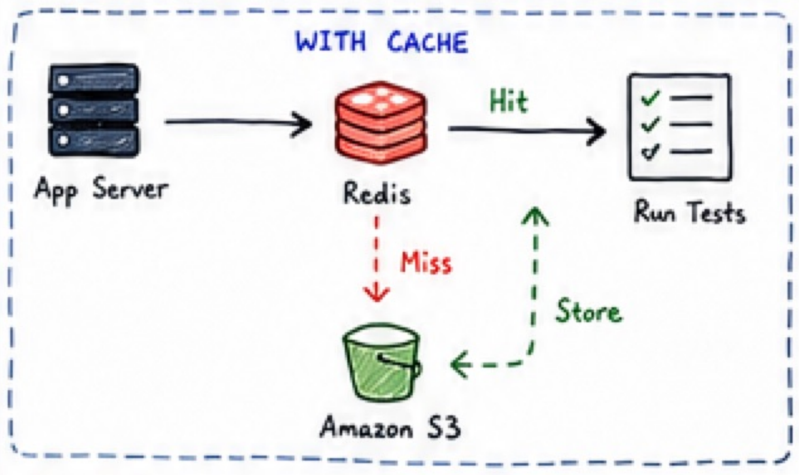
2 5 STEP CACHE DESIGN FRAMEWORK

1. **Should we cache?** Is caching beneficial for this use case?
2. **What should we cache?** What data to cache?
3. **Where should we cache?** Browser / CDN / App / Redis / Database
4. **How will we invalidate?** TTL / Events / Manual / Write Strategies
5. **Which eviction policy?** LRU / LFU / FIFO / Others

Think in terms of Read Path & Write Path

3 CASE STUDY – CODE JUDGE





Benefits

- ✓ Faster execution
- ✓ Lower network traffic
- ✓ Fewer S3 requests
- ✓ Saves bandwidth & cost

Key Takeaway

Cache large, immutable or rarely changing data close to the application.

4 CACHE LOCATIONS

- **Browser Cache** – Static files (HTML, CSS, JS, Images, Fonts)
- **CDN Cache** – Content close to users (Images, Videos, CSS, JS)
- **Application Cache** – In-memory cache inside the application
- **Redis / Distributed Cache** – Shared cache across multiple app servers
- **Database** – Source of truth

Closest to user – Faster ↑ Far from user – Slower ↓

5 CACHE INVALIDATION & TTL

Why invalidate?

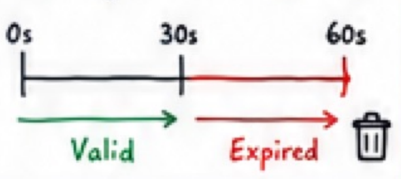
Data in cache becomes stale when source data changes.

Invalidation Strategies

- TTL (Time To Live)
- Event Based Invalidation
- Manual Invalidation
- Write Strategies (Through / Around / Back)

TTL Example

Set key = "A" TTL = 60s

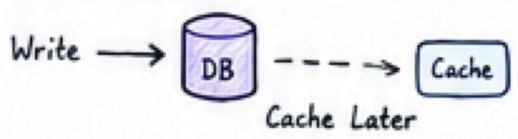


Consistency Models

- **Strong Consistency** – Always return latest data
- **Eventually Consistent** – Slightly stale data is acceptable

6 WRITE STRATEGIES

WRITE AROUND

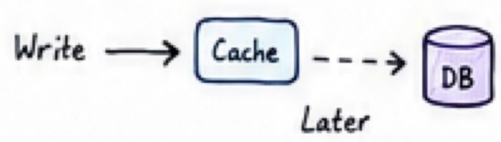


- ✓ Good for large writes
- ✗ Cache miss on next read

WRITE THROUGH

- ✓ Always up to date
- ✗ Higher latency (write path)

WRITE BACK (WRITE BEHIND)



- ✓ Very fast writes
- ✗ Risk of data loss (crash)

7 EVICTION POLICIES

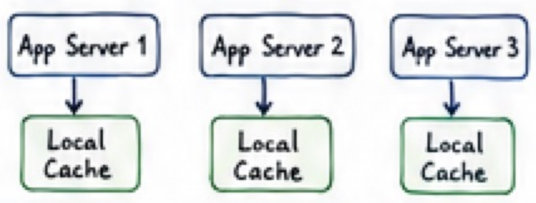
When cache is full, which item to remove?

- **FIFO** (First In First Out) – Remove the oldest item. 1 2 3 4 → 1
- **LIFO** (Last In First Out) – Remove the most recent item. 1 2 3 4 → 4
- **LRU** (Least Recently Used) – Remove least recently used. 1 2 3 4 2 → 5
- **LFU** (Least Frequently Used) – Remove least frequently used. A(5) B(1) C(3) → B(1)

LRU is the most widely used in real systems (Redis, Memcached).

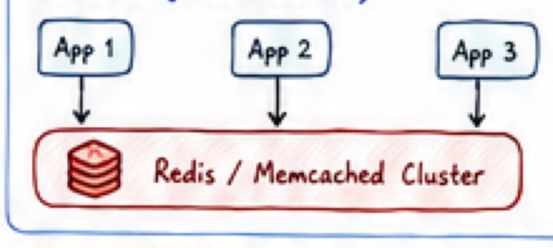
8 LOCAL vs GLOBAL CACHE

LOCAL (IN-MEMORY)



- Very fast
- Not shared
- Data duplication
- Good for small data

GLOBAL (DISTRIBUTED)



- Shared across apps
- Scalable
- Network overhead
- Good for medium to large data

Use Local Cache for session / request level data.
Use Global Cache for shared data.

9 REDIS QUICK OVERVIEW

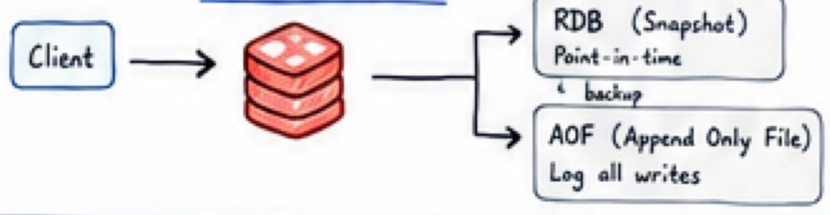
Key Features

- In-memory key-value store
- Extremely fast
- Supports rich data types
- Persistence (RDB, AOF)
- Replication & Cluster
- Pub/Sub, TTL, Transactions

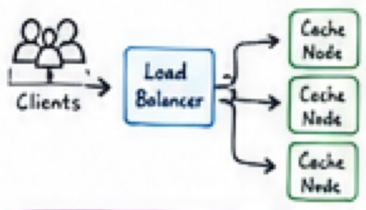
Common Data Types

- String (key → value)
- Hash (key → field-value)
- List (ordered list)
- Set (unique items)
- Sorted Set (score based)
- Bitmaps, HyperLogLog, etc.

Redis Architecture



10 LOAD BALANCER (FOR CACHE)



Should we use consistent hashing?

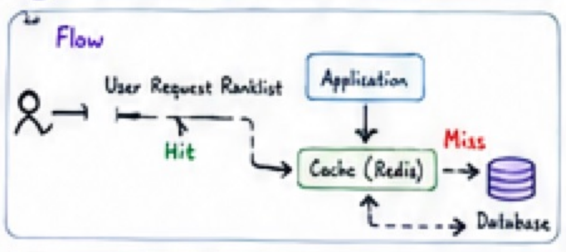
- Yes! For distributed cache (Redis Cluster / Memcached)
- Minimizes data movement when nodes are added or removed.

Multi-threading in LB?

Yes, helps handle more connections concurrently. Good for high throughput.

11 SCALER CONTEST LEADERBOARD (RANKLIST)

Flow



What to Cache?

- ☑ Ranklist data
- ☑ Top K users
- ☑ Pagination results
- ☑ Aggregated results

Important Questions

- How frequently is ranklist queried?
- Can ranklist change frequently?
- Can we calculate ranklist 200 times/sec?
- How large is the JSON response?
- Do we need immediate consistency?

Key Takeaway

Cache the frequently queried, expensive ranklist data. Use pagination + TTL + smart invalidation.

12 DOUBTS – QUICK NOTES

- Latency vs Throughput? → Cache reduces latency, increases throughput
- Cost of Network Transfer? → High! Always prefer caching near the user.
- Time to download testcases from S3? → High for large files. Cache them.
- Async IO vs Threading? → Async for IO-bound, Threading for CPU-bound.
- Ranklist calculation? → Precompute + cache or incremental update.

FAMOUS JOKE

Why do programmers prefer caching?

Because they hate waiting... for anything!

KEY TAKEAWAY

- ☑ Cache the right data
- ☑ Use the right strategy
- ☑ Keep it fast, consistent & scalable
- ☑ Always think about staleness & failures

RESOURCES (LEARN MORE)

- redis.io/learn/howtos/quick-start
- redis.io/university/
- redis.io/docs/latest/
- Redis Cheat Sheet (google it)

LEGEND

- Best Practice
- Warning / Don't
- Important
- Concept